

הסבר מלא על הקוד

Redact-FileContent.ps1

מסמך זה עובר על כל שורה ושורה של הסקריפט, לפי סדר הופעתן בקובץ, ומסביר בעברית מה כל בלוק קוד עושה בפועל וכיצד הוא משתלב בזרימה הכללית של הריצה - מקבלת הפרמטרים ועד לכתיבת הקבצים והדפסת הסיכום.

branch: claude/master

15 סעיפים

249 שורות קוד

מבנה הסקריפט לפי הסדר

```

1 <#
2 Redact-FileContent.ps1
3
4 Unified replacement/redaction engine for files, recursively through subfolders.
5 All modes below share the same file-processing, dry-run and backup logic.
6
7 1. Literal/regex replacements ($Replacements below) - always applied.
8     Each key is a regex pattern, each value is what to replace it with.
9     (Companion to Search-FilesForWord.ps1 - use that one first to preview matches.)
10
11 2. -AnonymizeSwitchNames - finds whitespace-delimited tokens that start with
12     "S"/"s" and end with "nn" (letters, digits, and special characters allowed
13     in between, e.g. S-DC1-01nn) and replaces each with a sequential generic
14     placeholder (SWITCH001, SWITCH002, ...).
15
16 3. -AnonymizeUsernames - finds tokens like "user22351" (literal "user" followed
17     directly by digits) and replaces each with a sequential placeholder
18     (USER001, USER002, ...).
19
20 4. -AnonymizeIPs - finds IPv4 addresses and replaces each with a sequential
21     placeholder (IP001, IP002, ...).
22
23 5. -StripPortDescriptions - removes the description text from interface lines
24     such as "GigabitEthernet1/0/1 - TO_SWITCH001", leaving just the port name
25     ("GigabitEthernet1/0/1"). Covers GigabitEthernet, TenGigabitEthernet,
26     TwentyFiveGigabitEthernet, FastEthernet and Ethernet ports.
27
28 For modes 2-4, the same original value always maps to the same placeholder
29 within a single run (each category has its own counter), but nothing is
30 persisted between runs - running the script again assigns fresh numbers.
31
32 All write modes support -DryRun (report what would change, touch nothing) and
33 -Backup (copy each modified file's original content aside before overwriting it).
34
35 USAGE EXAMPLES:
36
37 # Preview the literal replacements only, no files modified
38 .\Redact-FileContent.ps1 -Path "D:\\" -DryRun
39

```

```

40      # Apply literal replacements for real, keeping backups
41      .\Redact-FileContent.ps1 -Path "D:\" -Backup
42
43      # Anonymize switch names, usernames and IPs, and strip port descriptions
44      .\Redact-FileContent.ps1 -Path "D:\" -AnonymizeSwitchNames -AnonymizeUsernames -
AnonymizeIPs -StripPortDescriptions -Backup
45
46      # Anonymization only - skip the literal $Replacements table
47      .\Redact-FileContent.ps1 -Path "D:\" -AnonymizeIPs -Replacements @{} -DryRun
48
49      # Limit to specific file types
50      .\Redact-FileContent.ps1 -Path "D:\Logs" -Include *.log,*.txt -AnonymizeUsernames
-DryRun
51  #>

```

הבלוק בין `<#` ל-`#>` הוא הערת תיעוד (block comment) שלא מתבצעת בזמן ריצה. היא מסבירה למי שפותח את הקובץ, לפני שהוא בכלל מריץ אותו, מה הסקריפט עושה:

- הסקריפט הוא "מנוע החלפה/מחיקת מידע" (Redaction) אחד שרץ רקורסיבית על כל תיקיית משנה תחת נתיב נתון.
- קיימים 5 סוגי פעולות שאפשר להפעיל, כל אחת בנפרד או כמה יחד: החלפות מילוליות (תמיד פעיל), אנונימיזציה של שמות מתגים, אנונימיזציה של שמות משתמשים, אנונימיזציה של כתובות IP, ומחיקת תיאורי פורטים (Port Descriptions).
- מוסבר שהמיפוי בין ערך מקורי לשם הגנרי (למשל `S-DC1-01nn` -> `SWITCH001`) עקבי **רק בתוך הרצה אחת** ולא נשמר לריצות הבאות.
- בסוף מופיעות דוגמאות שימוש בפועל - איך מריצים את הסקריפט משורת הפקודה עם צירופי פרמטרים שונים.

```

53 [CmdletBinding(SupportsShouldProcess = $true)]
54 param(
55     # Root folder to process. Defaults to the current directory.
56     [string]$Path = ".",
57
58     # Optional file name filters (wildcards), e.g. *.txt, *.log, *.csv.
59     # Leave empty to process every file.
60     [string[]]$Include = @("*"),
61
62     # Ordered map of regex pattern -> replacement text, applied on every line.
63     [System.Collections.Specialized.OrderedDictionary]$Replacements = [ordered]@{
64         "ass" = "aaa"
65     },
66
67     # Anonymize switch/device name tokens matching -SwitchNamePattern.
68     [switch]$AnonymizeSwitchNames,
69     [string]$SwitchNamePattern = '(?<!\S)[Ss][^\s]*nn(?:!\S)',
70     [string]$SwitchNamePrefix = "SWITCH",
71
72     # Anonymize username tokens matching -UsernamePattern.
73     [switch]$AnonymizeUsernames,
74     [string]$UsernamePattern = '(?<!\S)user\d+(?:!\S)',
75     [string]$UsernamePrefix = "USER",
76
77     # Anonymize IPv4 addresses.
78     [switch]$AnonymizeIPs,
79     [string]$IPPattern = '(?<!\d)(?:(:?25[0-5]|2[0-4]\d|1?\d?\d)\.){3}(?:25[0-5]|
2[0-4]\d|1?\d?\d)(?:!\d)',
80     [string]$IPPrefix = "IP",
81
82     # Strip the description text off interface lines, e.g.
83     # "GigabitEthernet1/0/1 - TO_SWITCH001" -> "GigabitEthernet1/0/1".
84     [switch]$StripPortDescriptions,
85     [string]$PortDescriptionPattern = '^(?<port>(?:TwentyFiveGigabitEthernet|
TenGigabitEthernet|GigabitEthernet|FastEthernet|Ethernet)\d+(?:/\d+)*)\s*-\s*.*$',
86
87     # Report matches and what would change, without modifying any file.
88     [switch]$DryRun,
89
90     # Copy each file's original content into -BackupFolder before overwriting it.

```

```

91 [switch]$Backup,
92
93 # Folder to hold backups when -Backup is used. Defaults to a timestamped
94 # folder created next to -Path. Relative structure under -Path is preserved.
95 [string]$BackupFolder
96 )

```

זהו "החתימה" של הסקריפט - כל הפרמטרים שאפשר להעביר לו כשמריצים אותו. שורה 53
 ([CmdletBinding(SupportsShouldProcess = \$true)]) (הופכת את הסקריפט ל-cmdlet "אמיתי" של
 PowerShell, ובפרט מפעילה תמיכה ב-WhatIf (הדפסת "מה היה קורה" בלי לבצע בפועל) וב-Confirm .

פרמטר	מה הוא עושה
-Path	תיקיית השורש שעליה רצים. ברירת מחדל: התיקייה הנוכחית (.).
-Include	מסננת קבצים לפי שם/סיומת (למשל *.txt). ברירת מחדל: הכל.
-Replacements	טבלה מסודרת שבה כל מפתח הוא תבנית regex וכל ערך הוא הטקסט החלופי שלו. פעילה תמיד. ברירת המחדל היא ההחלפה המקורית "ass" -> "aaa".
- / -AnonymizeSwitchNames SwitchNamePattern / SwitchNamePrefix	הדגל שמפעיל אנונימיזציה שמות מתנים, ה-regex שמזהה אותם (מתחיל ב-S/s ומסתיים ב-hh), והקידומת לשם הגנרי (ברירת מחדל SWITCH).
- / -AnonymizeUsernames UsernamePrefix / UsernamePattern	אותו רעיון עבור שמות משתמשים (userNNNN), קידומת ברירת מחדל USER.
- / -IPPattern / -AnonymizeIPs IPPrefix	אותו רעיון עבור כתובות IPv4, קידומת ברירת מחדל IP.
- / -StripPortDescriptions PortDescriptionPattern	דגל שמפעיל מחיקת תיאורי פורטים, וה-regex שמזהה שורת פורט (ראה הסבר מורחב בסעיף 6 בהמשך).
-DryRun	מצב "הרצה יבשה" - רק מדווח מה היה משתנה, לא נוגע באף קובץ.
-BackupFolder / -Backup	האם לגבות כל קובץ שמשתנה לפני הכתיבה, ולאן.

שימו לב: פרמטרים בוליאניים כמו -DryRun ו-Backup מוגדרים כ-[switch] - כלומר אין צורך לתת להם ערך, מספיק לציין את השם שלהם בשורת ההרצה כדי "להדליק" אותם.

3 בדיקת תקינות הנתיב

```
98 if (-not (Test-Path -LiteralPath $Path)) {  
99     Write-Error "Path not found: $Path"  
100     return  
101 }
```

לפני שעושים כלום, בודקים עם `Test-Path` שהנתיב שהוזן ב- `-Path` באמת קיים על הדיסק. אם לא - מודפסת שגיאה עם `Write-Error` והפקודה `return` עוצרת את כל הסקריפט מיידית, כדי שלא ננסה להמשיך לעבוד על נתיב שלא קיים.

4 הכנת תיקיית הגיבוי (רק אם Backup- פעיל)

```
103 if ($Backup -and -not $BackupFolder) {  
104     $BackupFolder = Join-Path (Resolve-Path -LiteralPath $Path) "_backup_$(Get-Date -  
    Format 'yyyyMMdd_HH:mm:ss')"  
105 }
```

אם המשתמש ביקש `-Backup` אבל לא ציין באופן מפורש `-BackupFolder`, הסקריפט בונה שם תיקייה אוטומטי: תיקייה חדשה בשם `_backup_` ואחריה תאריך ושעה מדויקים (עד השנייה), שנוצרת ליד הנתיב המקורי. כך כל הרצה עם גיבוי מקבלת תיקיית גיבוי משלה, ולא דורסת גיבוי קודם.

```

107 $resolvedRoot = (Resolve-Path -LiteralPath $Path).ProviderPath
108 $allFiles = Get-ChildItem -Path $Path -Include $Include -Recurse -File -ErrorAction
    SilentlyContinue
109
110 $ignoreCase = [System.Text.RegularExpressions.RegexOptions]::IgnoreCase
111 $switchNameRegex = [regex]::new($SwitchNamePattern, $ignoreCase)
112 $usernameRegex = [regex]::new($UsernamePattern, $ignoreCase)
113 $ipRegex = [regex]::new($IPPattern)
114 $portDescriptionRegex = [regex]::new($PortDescriptionPattern, $ignoreCase)

```

- **שורה 107:** `$resolvedRoot` - הופך את `-Path` לנתיב מלא ומוחלט (ראו הסבר על `Resolve-Path` שכבר ניתן קודם בשיחה). זה נדרש כדי לחשב בהמשך את הנתיב היחסי של כל קובץ בתוך תיקיית הגיבוי.
- **שורה 108:** `Get-ChildItem` אוסף את כל הקבצים (הדגל `-File` מוציא תיקיות מהתוצאה) תחת `-Path`, רקורסיבית (`-Recurse`), עם הסינון של `-Include`. `-ErrorAction SilentlyContinue` גורם לסקריפט "לדלג בשקט" על תיקיות שאין הרשאה לגשת אליהן, במקום לקרוס.
- **שורות 110-114:** בונים מראש ארבעה אובייקטי `[regex]` קומפילציה - אחד לכל תבנית (מתגים, משתמשים, IP, תיאורי פורטים) - כדי לא לקמפל מחדש את אותו ה-regex בכל שורה בכל קובץ (יעילות). כולם (חוץ מ-IP) מקבלים את הדגל `IgnoreCase` כדי לתפוס גם אותיות גדולות וגם קטנות.

```

116 # Original value (as first encountered) -> generic placeholder, one map per category.
117 # Consistent for this run only - not persisted between runs.
118 $switchNameMap = [ordered]@{}
119 $usernameMap = [ordered]@{}
120 $ipMap = [ordered]@{}
121 $switchNameIndex = 1
122 $usernameIndex = 1
123 $ipIndex = 1
124
125 function Get-PlaceholderName {
126     param(
127         [string]$OriginalValue,
128         [System.Collections.Specialized.OrderedDictionary]$Map,
129         [string]$Prefix,
130         [string]$IndexVarName
131     )
132
133     if (-not $Map.Contains($OriginalValue)) {
134         $index = Get-Variable -Name $IndexVarName -Scope Script -ValueOnly
135         $Map[$OriginalValue] = "{0}{1:D3}" -f $Prefix, $index
136         Set-Variable -Name $IndexVarName -Scope Script -Value ($index + 1)
137     }
138     return $Map[$OriginalValue]
139 }

```

זהו ה"לב" של מנגנון האנונימיזציה העקבית. לכל אחת משלוש הקטגוריות (מתגים/משתמשים/IP) יש:

- **מפה (Ordered Dictionary)** - `$switchNameMap`, `$usernameMap`, `$ipMap` - ששומרת לכל ערך מקורי את השם הגנרי שהוקצה לו, לאורך כל ההרצה (על פני כל הקבצים, לא רק קובץ אחד).

- **מונה (Index)** - `$switchNameIndex` וכו' - כל אחד מתחיל מ-1, ועולה בכל פעם שנתקלים בערך חדש באותה קטגוריה.

הפונקציה `Get-PlaceholderName` (שורות 125-139) היא הליבה: מקבלת ערך מקורי, את המפה המתאימה, קידומת (כמו "SWITCH"), ואת שם המשתנה של המונה (כמחרוזת טקסט, לא את המספר עצמו!).

- אם הערך **עדיין לא** קיים במפה (`-not $Map.Contains(...)`) - קוראים לערך הנוכחי של המונה דרך `Get-Variable ... -Scope Script`, בונים ממנו שם חדש בפורמט `SWITCH001` (`D3`) = לפחות 3 ספרות, עם אפסים מובילים), שומרים אותו במפה, ומעלים את המונה ב-1 באמצעות `Set-Variable ... -Scope Script`.

- אם הערך **כבר** קיים במפה - פשוט מחזירים את השם הגנרי שכבר הוקצה לו בעבר באותה הרצה. כך מובטח שאותו S-DC1-01nn יקבל תמיד את אותו SWITCH001 בכל מקום שהוא מופיע.

השימוש ב- `Set-Variable -Scope Script / Get-Variable` (במקום פשוט `$SwitchNameIndex++`) הוא טריק שמאפשר לפונקציה אחת גנרית לעדכן את המונה הנכון מתוך שלוש אפשרויות, לפי שם המשתנה שהועבר אליה כפרמטר.

7 תחילת הלולאה הראשית - מעבר על כל קובץ

```

141 $changedCount = 0
142 $scannedCount = 0
143
144 foreach ($fileItem in $allFiles) {
145     $scannedCount++
146
147     $content = Get-Content -LiteralPath $fileItem.FullName -ErrorAction SilentlyContinue
148     if ($null -eq $content) {
149         continue
150     }
151
152     $fileMatchCount = 0
153     $updatedContent = foreach ($row in $content) {
154         $currentRow = $row

```

שני מונים גלובליים: `$changedCount` (כמה קבצים בפועל השתנו) ו-`$scannedCount` (כמה קבצים נסרקו בסך הכל), מאותחלים ל-0. הלולאה `foreach ($fileItem in $allFiles)` עוברת על כל קובץ שנאסף בשלב 5. עבור כל קובץ:

- המונה הכללי עולה ב-1.
- `Get-Content` קוראת את הקובץ **כמערך שורות** (`-ErrorAction SilentlyContinue` שוב מונע קריסה על קובץ בינארי/נעול/פגום).
- אם הקריאה נכשלה (`$content` ריק/null) - `continue` מדלג לקובץ הבא בלי לגעת בו.
- `$fileMatchCount` מתאפס לכל קובץ - זהו מונה ההתאמות שנמצאו **בקובץ הנוכחי** בלבד.

8 מעבר שורה-שורה: החלפות מילוליות (\$Replacements)

```
152     $fileMatchCount = 0
153     $updatedContent = foreach ($row in $content) {
154         $currentRow = $row
155
156         foreach ($search in $Replacements.Keys) {
157             $lineMatches = [regex]::Matches($currentRow, $search).Count
158             if ($lineMatches -gt 0) {
159                 $fileMatchCount += $lineMatches
160                 $currentRow = $currentRow -replace $search, $Replacements[$search]
161             }
162         }
163     }
```

עבור כל שורה בקובץ (`foreach ($row in $content)`), עוברים על כל תבנית ב- `$Replacements` (ברירת מחדל: רק `"ass" -> "aaa"` , אבל אפשר להעביר טבלה אחרת עם כמה זוגות שרוצים). לכל תבנית: סופרים כמה פעמים היא מופיעה בשורה עם `[regex]::Matches(...).Count` , מוסיפים את המספר הזה למונה הכולל של הקובץ, ורק אם יש לפחות התאמה אחת - מבצעים בפועל את ההחלפה עם האופרטור `-replace` .

9 מחיקת תיאור פורט (-StripPortDescriptions)

```
164     if ($StripPortDescriptions -and $portDescriptionRegex.IsMatch($currentRow)) {
165         $fileMatchCount++
166         $currentRow = $portDescriptionRegex.Replace($currentRow, '${port}')
167     }
```

אם הדגל `-StripPortDescriptions` פעיל, בודקים אם השורה כולה תואמת את התבנית `$PortDescriptionPattern` (למשל `GigabitEthernet1/0/1 - TO_SWITCH001`). אם כן - מעלים את מונה ההתאמות ב-1, ומחליפים את כל השורה בקבוצת ה- `(?<port>...)` שנתפסה מתוך ה-regex (כלומר משאירים רק את שם הפורט, `GigabitEthernet1/0/1` , ומוחקים לגמרי את המקף ואת הטקסט שאחריו).

חשוב שהשלב הזה מתבצע לפני שלושת שלבי האנונימיזציה הבאים - כך אם גם `-AnonymizeSwitchNames` וגם `-StripPortDescriptions` פעילים יחד, קודם נמחק ה-description (שכולל למשל `TO_SWITCH001`), ורק מה שנשאר בשורה (שם הפורט בלבד) עובר הלאה לבדיקת האנונימיזציה.

```

169         if ($AnonymizeSwitchNames) {
170             $currentRow = $switchNameRegex.Replace($currentRow, {
171                 param($match)
172                 $script:fileMatchCount++
173                 Get-PlaceholderName -OriginalValue $match.Value -Map $switchNameMap -
Prefix $SwitchNamePrefix -IndexVarName "switchNameIndex"
174             })
175         }
176
177         if ($AnonymizeUsernames) {
178             $currentRow = $usernameRegex.Replace($currentRow, {
179                 param($match)
180                 $script:fileMatchCount++
181                 Get-PlaceholderName -OriginalValue $match.Value -Map $usernameMap -
Prefix $UsernamePrefix -IndexVarName "usernameIndex"
182             })
183         }
184
185         if ($AnonymizeIPs) {
186             $currentRow = $ipRegex.Replace($currentRow, {
187                 param($match)
188                 $script:fileMatchCount++
189                 Get-PlaceholderName -OriginalValue $match.Value -Map $ipMap -Prefix
$IPPrefix -IndexVarName "ipIndex"
190             })
191         }
192
193         $currentRow
194     }

```

שלושה בלוקים כמעט זהים במבנה - אחד לכל קטגוריה (מתגים, משתמשים, IP). כל בלוק פעיל רק אם הדגל המתאים דלוק (`if ($AnonymizeSwitchNames)` וכו'). בכל בלוק:

- קוראים ל- `$xxxRegex.Replace($currentRow, {...})` - הפעולה `.Replace` של אובייקט ה-regex מקבלת **סקריפט-בלוק** (**MatchEvaluator**) במקום מחרוזת קבועה - כלומר עבור **כל** התאמה שנמצאת בשורה, מריצים פונקציה שמחליטה מה יהיה הטקסט החלופי, במקום להחליף תמיד באותו טקסט קבוע.
- בתוך אותו סקריפט-בלוק: `$script:fileMatchCount++` מעלה את מונה ההתאמות של הקובץ, ואז `Get-PlaceholderName` (שהוסברה בסעיף 6) מוחזרת כתוצאה - כלומר הטקסט המקורי (למשל `S-DC1-01nn`) מוחלף בשם הגנרי העקבי (`SWITCH001`).

שלושת השלבים רצים לפי הסדר: קודם שמות מתגים, אחר כך שמות משתמשים, ולבסוף כתובות IP - כל אחד פועל על התוצאה של הקודם לו באותה שורה.

11 סיום הלולאה הפנימית וההחלטה אם לכתוב לקובץ

```
193     $currentRow
194 }
195
196 if ($fileMatchCount -eq 0) {
197     continue
198 }
199
200 $changedCount++
201
202 if ($DryRun) {
203     Write-Host "[DryRun] Would update: $($fileItem.FullName) ($fileMatchCount
match(es))" -ForegroundColor DarkYellow
204     continue
205 }
206
207 if (-not $PSCmdlet.ShouldProcess($fileItem.FullName, "Replace $fileMatchCount
match(es)")) {
208     continue
209 }
```

- **שורה 193:** השורה המעודכנת (`$currentRow`), אחרי כל ההחלפות שהופעלו) מוחזרת מתוך הלולאה הפנימית ונאספת לתוך `$updatedContent` - כך שבסוף יש לנו את כל הקובץ מחדש, שורה אחר שורה, עם כל השינויים.
- **שורות 196-198:** אם אף שורה בקובץ לא השתנתה (`$fileMatchCount -eq 0`) - מדלגים לקובץ הבא. הקובץ לא נכתב מחדש, לא מגובה, ולא נספר כ"שונה".
- **שורה 200:** אחרת - מונה הקבצים ששוננו עולה ב-1.
- **שורות 202-205:** במצב `-DryRun` - רק מודפסת שורת דיווח כתומה עם שם הקובץ ומספר ההתאמות, ומדלגים הלאה בלי לגעת בקובץ בפועל.
- **שורות 207-209:** `$PSCmdlet.ShouldProcess(...)` הוא המנגנון הפנימי של PowerShell שמפעיל את `-Confirm /WhatIf` (שהופעל בשורה 53). אם המשתמש הריץ עם `-WhatIf`, השורה הזו תחזיר `$false` והקובץ לא ייכתב - בלי צורך בקוד נוסף.

12 ביצוע הגיבוי בפועל

```
211     if ($Backup) {
212         $relativePath =
213             $fileItem.FullName.Substring($resolvedRoot.Length).TrimStart('\','/')
214         $backupPath = Join-Path $BackupFolder $relativePath
215         $backupDir = Split-Path -Path $backupPath -Parent
216         if (-not (Test-Path -LiteralPath $backupDir)) {
217             New-Item -ItemType Directory -Path $backupDir -Force | Out-Null
218         }
219         Copy-Item -LiteralPath $fileItem.FullName -Destination $backupPath -Force
220     }
```

אם `-Backup` פעיל: מחשבים את הנתבי **היחסי** של הקובץ (המסלול שלו ביחס לתיקיית השורש `$resolvedRoot` שחושבה בשורה 107), ובונים ממנו נתיב מקביל בתוך תיקיית הגיבוי - כך שמבנה התיקיות המקורי נשמר גם בגיבוי (קובץ ב- `D:\Configs\SwitchA.txt` יגובה ל- `<BackupFolder>\Configs\SwitchA.txt` ולא לתיקייה שטוחה אחת). אם תיקיית היעד עדיין לא קיימת - `New-Item -ItemType Directory` יוצרת אותה (כולל כל תיקיות הביניים החסרות, בזכות `-Force`). לבסוף `Copy-Item` מעתיקה את הקובץ **המקורי** (לפני השינוי) למיקום הגיבוי.

13 כתיבת הקובץ המעודכן וסגירת הלולאה

```
221     Write-Host "Working on: $($fileItem.FullName) ($fileMatchCount match(es))" -
222         ForegroundColor DarkYellow
223     $updatedContent | Set-Content -LiteralPath $fileItem.FullName
224 }
```

מודפסת שורת סטטוס צהובה עם שם הקובץ ומספר ההתאמות שנמצאו בו, ואז `Set-Content` כותבת את `$updatedContent` (התוכן החדש שנבנה בסעיפים 8-11) בחזרה לאותו קובץ, ודורסת את התוכן הישן. זהו הרגע היחיד שבו קובץ באמת משתנה על הדיסק.

```

225 Write-Host ""
226 if ($DryRun) {
227     Write-Host "Dry run complete. $changedCount of $scannedCount file(s) would be
updated." -ForegroundColor Green
228 } else {
229     Write-Host "Complete successfully! $changedCount of $scannedCount file(s) updated."
-ForegroundColor Green
230     if ($Backup) {
231         Write-Host "Backups saved under: $BackupFolder" -ForegroundColor Green
232     }
233 }

```

אחרי שהלולאה על כל הקבצים הסתיימה, מודפס סיכום: במצב `-DryRun` - כמה קבצים היו משתנים מתוך הסך הכל שנסקרו. אחרת - כמה קבצים **אכן** שוננו, ואם היה גיבוי - גם הנתיב המלא לתיקיית הגיבוי שבה אפשר למצוא את הקבצים המקוריים.

```

235 function Write-NameMapping {
236     param([string]$Title, [System.Collections.Specialized.OrderedDictionary]$Map)
237
238     if ($Map.Count -eq 0) {
239         return
240     }
241     Write-Host "`n$Title mapping for this run:" -ForegroundColor Cyan
242     $Map.GetEnumerator() | ForEach-Object {
243         Write-Host ("  {0}  ->  {1}" -f $_.Key, $_.Value)
244     }
245 }
246
247 if ($AnonymizeSwitchNames) { Write-NameMapping -Title "Switch name" -Map
    $switchNameMap }
248 if ($AnonymizeUsernames) { Write-NameMapping -Title "Username" -Map $usernameMap }
249 if ($AnonymizeIPs) { Write-NameMapping -Title "IP address" -Map $ipMap }

```

הפונקציה `Write-NameMapping` (שורות 235-245) מקבלת כותרת ומפה, ואם יש בה לפחות ערך אחד - מדפיסה כותרת כחולה ואז שורה לכל ערך מקורי, עם השם הגנרי שהוקצה לו, שנוצר באותה הרצה (לדוגמה: `S-DC1-01nn` `-> SWITCH001`).

בשלוש השורות האחרונות של הסקריפט (247-249) קוראים לפונקציה הזו פעם אחת לכל קטגוריה שהייתה פעילה - כדי שבסוף ההרצה יהיה למפעיל תיעוד מלא של מה הוחלף במה, גם אם המיפוי עצמו לא נשמר לקובץ ולא ישמש בהרצה הבאה.